# Inyección SQL en el login

## Resultado de la práctica inicial

Se confirmó la omisión de la validación de contraseña. El usuario **admin** con contraseña **incorrecta** recibió HTTP 401; al enviar **admin' --** con la misma contraseña, el endpoint respondió HTTP 200 y **ok: true, user: admin**.

## Objetivo y alcance

Demostrar y explicar una inyección SQL en el login educativo recién incorporado. Las pruebas se realizaron exclusivamente contra **http://127.0.0.1:3000/login-vulnerable**, en la copia local, con los usuarios de demostración y una base SQLite en memoria. No se enviaron pruebas a GitHub ni a servidores externos.

## Entorno y preparación

Aplicación Node.js, Express y sqlite3. Se instalaron las dependencias con **npm ci**. El servidor se ejecutó con un envoltorio temporal que limita la escucha a 127.0.0.1 y guarda las consultas en servidor.log, sin modificar el código del login.

Código evaluado: **login/server-vulnerable.js**, líneas 26-38. Revisión local: `fc48115f39d326d76ec9ba991be92494f1111649`.

## Procedimiento

1. Enviar admin / incorrecta mediante el formulario y capturar el rechazo.
2. Enviar admin' -- (con espacio final) / incorrecta y capturar la respuesta.
3. Repetir por HTTP y registrar estado y cuerpo JSON.
4. Contrastar la consulta del servidor y formular la corrección.

**Actualización:** las capturas de las páginas 2 y 3 corresponden a la versión vulnerable. Las páginas 4 a 6 documentan la corrección y las pruebas. Las páginas 7 y 8 detallan cuatro payloads funcionales en la versión anterior, por qué funcionan, su rechazo con parámetros y las conclusiones. El endpoint devuelve JSON, sin sesiones ni panel privado.

# Evidencia 1 / Control de acceso

## Contraseña incorrecta: rechazo esperado

```
Usuario: admin
Contraseña: incorrecta
POST /login-vulnerable
HTTP 401
{"ok": false, "message": "Credenciales inválidas"}
```

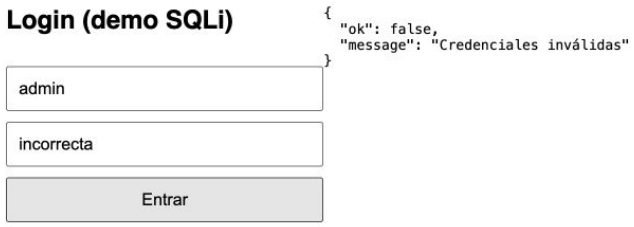


**Figura 1.** Captura real del formulario después de enviar credenciales inválidas. La respuesta visible indica ok: false. El estado HTTP 401 se verificó adicionalmente en la repetición registrada en respuestas-http.json.

## Control positivo

La repetición con admin / admin devolvió HTTP 200 y {"ok": true, "user": "admin"}. Este control confirma el comportamiento normal del endpoint con las credenciales precargadas.

# Evidencia 2 / Bypass confirmado

Entrada de usuario: **admin' --** (espacio final). Contraseña: **incorrecta**. Respuesta HTTP verificada: **200**.

**Figura 2.** Captura real del formulario: la misma contraseña incorrecta produce ahora ok: true y user: admin. Se conserva la captura completa tal como la entregó el navegador.

## Consulta observada en el registro

```
SELECT * FROM users WHERE username = 'admin' -- '
AND password = 'incorrecta'
```

La consulta del registro es una sola línea (aquí se divide por espacio). La comilla cierra el literal del usuario y **--** comenta el resto, incluida la condición de contraseña. La búsqueda efectiva selecciona al usuario admin y db.get devuelve su fila.

# Causa, corrección y evidencias

## Causa e impacto observado

La línea 30 inserta entradas del cliente en el texto SQL mediante interpolación. Los datos alteran la estructura de la consulta. En este laboratorio, el impacto demostrado es la suplantación de admin ante el endpoint sin su contraseña. No se probó extracción masiva ni modificación de datos.

## Corrección aplicada

Se reemplazó la interpolación por parámetros enlazados en login/server-vulnerable.js. La consulta actual recibe el usuario y la contraseña como valores separados:

```
const query =
  "SELECT username FROM users" +
  " WHERE username = ? AND password = ?";
db.get(query, [username, password], (err, row) => {
  // Manejo de error, éxito y rechazo.
});
```

Con parámetros, el usuario se trata como un valor. La documentación oficial de SQLite describe estos marcadores y el enlace de valores en tiempo de ejecución: https://www.sqlite.org/lang_expr.html#parameters

**Estado de la corrección:** aplicada y verificada. Admin / incorrecta devuelve 401; admin / admin, 200; admin' -- / incorrecta, 401. También se parametrizaron las inserciones de demostración y se validan campos de tipo cadena no vacíos.

Se eliminó el registro de consultas con contraseñas, se usa una respuesta genérica ante errores de base de datos y el servidor escucha en 127.0.0.1. Se conserva el nombre de la ruta /login-vulnerable para mantener compatible el formulario.

## Archivos de evidencia

En la carpeta **evidencias/**: 01-rechazo.jpg, 02-inyeccion.jpg, respuestas-http.json y servidor.log. Las capturas documentan las pruebas del formulario; el JSON contiene las repeticiones HTTP con marcas de tiempo UTC. El registro permite verificar el SQL recibido.

## Conclusión

La comparación inicial confirmó la inyección por concatenación. Después del cambio, los intentos probados se tratan como datos y no omiten la validación de contraseña. Las contraseñas de demostración siguen en texto claro; un uso real requiere hash de contraseñas y gestión de sesiones.

# Tipos de inyección SQL

La tabla reúne técnicas, objetivos y formas de activación que pueden combinarse. La peligrosidad expresa impacto potencial: es una estimación educativa, no una puntuación CVSS ni una clasificación oficial. Depende de permisos, datos y controles.

| Tipo o técnica | Descripción | Peligrosidad potencial |
|---|---|---|
| Omisión de autenticación | Altera condiciones o introduce comentarios para omitir la contraseña. | Crítica al suplantar cuentas privilegiadas. |
| Basada en UNION | Combina resultados de la consulta original con otra consulta. | Alta a crítica por exposición de datos. |
| Basada en errores | Usa errores para revelar estructura o información. | Alta a crítica según los datos revelados. |
| Ciega por booleanos | Deduce información comparando respuestas a condiciones verdaderas y falsas. | Alta a crítica según lo deducible. |
| Ciega por tiempo | Deduce información mediante demoras condicionadas. | Alta a crítica; posible impacto en disponibilidad. |
| Consultas apiladas | Intenta ejecutar sentencias adicionales si motor y controlador lo permiten. | Crítica si permite modificar o borrar datos. |
| Fuera de banda | Recupera resultados por otro canal si motor y red lo permiten. | Alta a crítica por extracción de datos. |
| De segundo orden | Una entrada almacenada se reutiliza después en SQL construido de forma insegura. | Alta a crítica según la operación posterior. |

No todas estas técnicas aplican a este login. La práctica inicial confirmó la omisión de contraseña mediante comentario. Las pruebas posteriores verifican el rechazo de entradas concretas; no constituyen una explotación de todas las técnicas de la tabla.

Fuentes: OWASP Web Security Testing Guide, sección de SQL Injection; OWASP SQL Injection y SQL Injection Prevention Cheat Sheet. Enlaces en la página 6.

# Verificación posterior al cambio

Las pruebas automatizadas de login/test-login.cjs inician y cierran un servidor en 127.0.0.1 con puerto asignado por el sistema. Ejecución: **cd login**, después **npm test**. Resultado: 12 casos HTTP aprobados; el ejecutor cuenta 13 pruebas al incluir la contenedora.

| Caso | HTTP esperado y observado |
|---|---|
| Admin válido / Juan válido | 200 / 200 |
| Admin con contraseña incorrecta | 401 |
| Usuario admin' -- con contraseña incorrecta | 401 |
| Tautología en usuario / en contraseña | 401 / 401 |
| Consulta apilada enviada como texto | 401 |
| Usuario con comilla literal | 401 |
| Campos ausentes / tipo inválido / cadena vacía | 400 / 400 / 400 |
| Admin válido después de los intentos | 200 |

## Interpretación y alcance

El payload que antes devolvía admin ahora es rechazado. Los accesos legítimos siguen funcionando y la tabla de usuarios permanece utilizable tras los intentos. No se ejecutaron pruebas de extracción por UNION, errores, tiempo, fuera de banda o segundo orden.

La salida de la verificación se conserva en **evidencias/pruebas-parametrizacion.txt**. Las capturas y respuestas-http.json originales documentan el estado anterior; no deben interpretarse como resultados del código corregido.

## Referencias

OWASP Web Security Testing Guide: SQL Injection
OWASP: SQL Injection e impacto
OWASP: SQL Injection Prevention Cheat Sheet
SQLite: parámetros en expresiones SQL

# Payloads que funcionan y su causa

Se ejecutó la misma petición POST /login-vulnerable contra dos servidores locales aislados: la revisión vulnerable fc48115 y el código parametrizado actual. Se conservan las consultas originales; el envoltorio solo asigna puertos dinámicos en 127.0.0.1. Los servidores se cerraron al terminar.

| Caso | Usuario exacto | Contraseña exacta | Vulnerable | Corregida |
|---|---|---|---|---|
| P1 | admin' -- | incorrecta | 200 / admin | 401 |
| P2 | juan' -- | incorrecta | 200 / juan | 401 |
| P3 | ' OR 1=1 -- | incorrecta | 200 / admin | 401 |
| P4 | admin | ' OR '1'='1 | 200 / admin | 401 |

P1, P2 y P3 llevan un espacio después de **--**. En todos los casos la respuesta vulnerable fue **ok: true** y la corregida **ok: false**. Los valores son datos de laboratorio.

## P1 y P2: comentario tras un usuario conocido

```
SELECT * FROM users WHERE username = 'admin' -- '
AND password = 'incorrecta'
```

La comilla cierra el nombre del usuario. El comentario -- anula el resto de la línea, incluida la comparación de contraseña. La consulta efectiva busca solo admin. P2 hace lo mismo con juan y devuelve juan. El salto de línea mostrado es únicamente de presentación; el SQL original es una sola línea.

## P3: condición verdadera en el usuario

```
SELECT * FROM users WHERE username = '' OR 1=1 -- '
AND password = 'incorrecta'
```

El predicado OR 1=1 es verdadero y el comentario elimina la condición de contraseña. Se seleccionan filas sin validar credenciales. db.get devuelve una sola fila; en esta ejecución fue admin. Al no existir ORDER BY, ese orden no debe darse por garantizado.

## P4: condición verdadera en la contraseña

```
SELECT * FROM users WHERE username = 'admin'
AND password = '' OR '1'='1'
```

La comilla de la entrada cierra el literal de contraseña. La comparación OR '1'='1' siempre es verdadera. AND se evalúa antes que OR, por lo que el predicado completo acepta filas aunque la contraseña no coincida. En la prueba, db.get devolvió admin.

# Parametrización y conclusiones

## Corrección de la petición SQL

El servidor mantiene fijo el texto SQL y enlaza ambos campos con sqlite3. La petición HTTP continúa enviando JSON; la protección se aplica al construir y ejecutar la consulta en el servidor. Fragmento aplicado en login/server-vulnerable.js:

```
const query =
  "SELECT username FROM users" +
  " WHERE username = ? AND password = ?";

db.get(query, [username, password], (err, row) => {
  if (err) return res.status(500).json({
    ok: false, message: "Error interno del servidor"
  });
  if (row) return res.json({ ok: true, user: row.username });
  res.status(401).json({
    ok: false, message: "Credenciales inválidas"
  });
});
```

Los signos ? son marcadores. El arreglo [username, password] proporciona los valores, sin concatenarlos al SQL. Así, admin' -- se busca como nombre completo y ' OR '1'='1 como contraseña literal: las comillas, OR y -- contenidos en los valores no se interpretan como estructura de la consulta.

## Resultados y evidencia reproducible

Ejecutar **python3 output/pdf/comparar_payloads.py** desde la raíz del repositorio. Evidencia: **evidencias/comparacion-payloads.json**, con fecha UTC, entradas y respuestas. Se aprobaron 12 comparaciones HTTP: seis por versión. Los cuatro payloads pasan de 200 a 401; el control negativo sigue en 401 y el positivo sigue en 200. Esta comparación complementa las pruebas de npm test descritas en la página 6.

## Conclusiones

1. Los cuatro payloads funcionaron porque las entradas del cliente se incorporaban al código SQL y cambiaban sus condiciones.
2. La parametrización aplicada impide esa alteración en los casos comprobados, manteniendo el inicio de sesión con credenciales válidas.
3. Validar tipos y evitar revelar errores o contraseñas complementa la corrección; no sustituye los parámetros.
4. El resultado demuestra aceptación de identidad en JSON, sin acreditar una sesión ni acceso a recursos privados. El laboratorio conserva contraseñas en texto claro y requiere hash de contraseñas y sesiones antes de un uso real.